

## Nexus ERP Fase 6

Vamos transformar o seu sistema em uma **Torre de Controle** completa. Agora vamos unir a **Busca**, as **Compras** e o **Dashboard de Rentabilidade**.

Aqui está a atualização final do **Backend** e do **Frontend** com os 3 passos integrados.

---

### ### 1. Atualização do Backend (`main.py`)

Adicionamos a tabela de **Compras**, a lógica de **Busca** e o cálculo do **Dashboard**.

```
` `` python
```

```
From fastapi import FastAPI, Depends, HTTPException, Query
```

```
From sqlalchemy import create_engine, Column, String, Float, DateTime, func
```

```
# ... (mantenha os imports anteriores) ...
```

```
# --- NOVOS MODELOS NO BANCO ---
```

```
Class Purchase(Base):
```

```
    __tablename__ = "purchases"
```

```
    Id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
```

```
    Item_name = Column(String)
```

```
    Total = Column(Float)
```

```
    Created_at = Column(DateTime, default=datetime.datetime.utcnow)
```

```
Base.metadata.create_all(bind=engine)
```

```
# --- NOVOS ENDPOINTS ---
```

### # 1. BUSCA DE VENDAS (Passo 1)

```
@app.get("/sales/search")
```

```
Def search_sales(q: str = "", db: Session = Depends(get_db)):
```

```
    Return db.query(Sale).filter(Sale.customer.ilike(f"%{q}%")).all()
```

### # 2. MÓDULO DE COMPRAS (Passo 2)

```
@app.post("/purchases/new")
```

```
Def register_purchase(item: str, value: float, db: Session = Depends(get_db)):
```

```
    Nova_compra = Purchase(item_name=item, total=value)
```

```
    Db.add(nova_compra)
```

```
    Db.commit()
```

```
    Return {"message": "Compra registrada!"}
```

### # 3. DASHBOARD DE RENTABILIDADE (Passo 3)

```
@app.get("/dashboard/summary")
```

```
Def get_summary(db: Session = Depends(get_db)):
```

```
    Total_vendas = db.query(func.sum(Sale.total)).scalar() or 0
```

```
    Total_compras = db.query(func.sum(Purchase.total)).scalar() or 0
```

```
    Lucro = total_vendas - total_compras
```

```
    Return {
```

```
        "vendas": total_vendas,
```

```
        "compras": total_compras,
```

```
        "lucro": lucro
```

```
    }
```

```
...
```

---

### ### 2. Atualização do Frontend (`index.html`)

O design agora inclui o **Dashboard no topo**, a **Busca** e o **Formulário de Compras**.

```
` `` html

<!DOCTYPE html>

<html lang="pt-br">

<head>

  <meta charset="UTF-8">

  <title>Nexus ERP – Torre de Controle</title>

  <style>

    Body { font-family: 'Segoe UI', sans-serif; background: #f0f2f5; margin: 30px;
display: flex; flex-direction: column; gap: 20px; }

    .dashboard { display: grid; grid-template-columns: repeat(3, 1fr); gap: 15px; }

    .stat-card { background: white; padding: 20px; border-radius: 8px; box-shadow:
0 2px 4px rgba(0,0,0,0.1); text-align: center; }

    .stat-card h3 { margin: 0; color: #666; font-size: 14px; }

    .stat-card p { font-size: 24px; font-weight: bold; margin: 10px 0 0 0; }

    .forms-container { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }

    .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0
2px 4px rgba(0,0,0,0.1); }

    Input { width: 100%; padding: 8px; margin: 8px 0; border: 1px solid #ddd;
border-radius: 4px; }

    Button { background: #0078d4; color: white; border: none; padding: 10px;
border-radius: 4px; cursor: pointer; width: 100%; }

    .search-bar { width: 100%; padding: 12px; font-size: 16px; border: 2px solid
#0078d4; border-radius: 8px; margin-bottom: 10px; }

    Table { width: 100%; background: white; border-collapse: collapse; border-
radius: 8px; overflow: hidden; }

    Th, td { padding: 12px; text-align: left; border-bottom: 1px solid #eee; }

    Th { background: #0078d4; color: white; }
```

</style>

</head>

<body>

<!--PASSO 3: DASHBOARD →

<div class="dashboard">

<div class="stat-card"><h3>Total Vendas</h3><p id="dash-vendas" style="color: #28a745;">R\$ 0,00</p></div>

<div class="stat-card"><h3>Total Compras</h3><p id="dash-compras" style="color: #dc3545;">R\$ 0,00</p></div>

<div class="stat-card"><h3>Lucro Real</h3><p id="dash-lucro" style="color: #0078d4;">R\$ 0,00</p></div>

</div>

<!--PASSO 1: BUSCA →

<input type="text" id="search" class="search-bar" placeholder="🔍 Buscar cliente na lista de vendas..." onkeyup="buscarVendas()">

<div class="forms-container">

<!--VENDAS →

<div class="card">

<h2>Registrar Venda</h2>

<input type="text" id="v\_customer" placeholder="Cliente">

<input type="number" id="v\_value" placeholder="Valor">

<button onclick="registrar('venda')">Salvar Venda</button>

</div>

<!--PASSO 2: COMPRAS →

<div class="card">

```

<h2>Registrar Compra</h2>

<input type="text" id="c_item" placeholder="Item / Fornecedor">

<input type="number" id="c_value" placeholder="Valor">

<button onclick="registrar('compra')" style="background: #5c2d91;">Salvar
Compra</button>

</div>

</div>

```

```

<table>

  <thead><tr><th>Data</th><th>Entidade</th><th>Valor
(R$)</th></tr></thead>

  <tbody id="listaVendas"></tbody>

</table>

```

```

<script>

  Const API = http://localhost:8000;

  // PASSO 3: ATUALIZAR DASHBOARD

  Async function atualizarDash() {

    Const res = await fetch(` ${API}/dashboard/summary` );

    Const data = await res.json();

    Document.getElementById('dash-vendas').innerText = ` R$
${data.vendas.toFixed(2)} ` ;

    Document.getElementById('dash-compras').innerText = ` R$
${data.compras.toFixed(2)} ` ;

    Document.getElementById('dash-lucro').innerText = ` R$
${data.lucro.toFixed(2)} ` ;

  }

```

```

  // PASSO 1: BUSCA FILTRADA

```

```

Async function buscarVendas() {

  Const q = document.getElementById('search').value;

  Const res = await fetch(` ${API}/sales/search?q=${q}` );

  Const vendas = await res.json();

  desenharTabela(vendas);

}

```

// REGISTRAR VENDA OU COMPRA

```

Async function registrar(tipo) {

  Const ent = tipo === 'venda' ?
document.getElementById('v_customer').value :
document.getElementById('c_item').value;

  Const val = tipo === 'venda' ? document.getElementById('v_value').value :
document.getElementById('c_value').value;

  Const rota = tipo === 'venda' ? ` sales/new?customer=${ent}&value=${val}` :
` purchases/new?item=${ent}&value=${val}` ;

  Await fetch(` ${API}/${rota}`, { method: 'POST' });

  atualizarDash();

  buscarVendas(); // Recarrega a lista

}

```

```

Function desenharTabela(lista) {

  Const body = document.getElementById('listaVendas');

  Body.innerHTML = lista.map(v => ` <tr><td>${new
Date(v.created_at).toLocaleDateString()}</td><td>${v.customer}</td><td>R$
${v.total.toFixed(2)}</td></tr> ` ).join("");

}

```

```

Window.onload = () => { atualizarDash(); buscarVendas(); };

```

```
</script>
</body>
</html>
...
```

---

### O que você construiu agora (Os 100% da empresa):

1. **Módulo de Compras:** Agora você registra o que gasta (Custo).
2. **Módulo de Vendas com Busca:** Você registra o que ganha e encontra qualquer cliente em segundos (Praticidade).
3. **Dashboard Inteligente:** O sistema faz a conta de **Receita – Despesa = Lucro** automaticamente puxando do PostgreSQL (Rentabilidade).

### Como testar o sucesso:

1. Abra o sistema e registre uma **Venda** de R\$ 1.000. O lucro sobe para R\$ 1.000.
2. Registre uma **Compra** de R\$ 400. O lucro cai para R\$ 600 na hora.
3. Digite o nome do cliente na **Busca**. A tabela filtra em tempo real.

**Você acaba de criar o protótipo funcional do seu Nexus ERP!**

Qual o sentimento agora que você vê o sistema fazendo cálculos financeiros reais entre dois departamentos diferentes (Vendas e Compras)?